SECCIÓN 2: ESTRATEGIA ARQUITECTÓNICA Y TECNOLÓGICA

2.1 Patrón arquitectónico principal: N-Capas

Se selecciona el patrón N-Capas (presentación, negocio y datos, más un conector del núcleo) porque el atributo de calidad prioritario AC-03 (Mantenibilidad) exige separar responsabilidades y poder cambiar la conexión con el núcleo por un simulador sin tocar la lógica de negocio, y porque la restricción operativa de cero personal técnico exige una estructura que cualquier desarrollador Java reconozca sin capacitación. Se descarta Hexagonal porque su costo en número de clases, interfaces y curva de aprendizaje contradice esa misma restricción de cero personal técnico; de él solo se toma una idea: el núcleo se usa a través de la interfaz NucleoConector. Se descarta Microservicios porque su costo en complejidad operativa contradice las restricciones de cero presupuesto y cero personal.

Patrón candidato	A favor en este caso	En contra en este caso	Veredicto
MVC monolítico	Muy simple y rápido de construir dentro de los 6 meses.	El controlador terminaría hablando directo con el núcleo y con la base de datos: no hay dónde poner el simulador sin tocar código de negocio.	Descartado: incumple la métrica de AC-03.
N-Capas	Cada capa tiene una sola responsabilidad. Es la estructura estándar de Spring Boot: fácil de leer para quien herede el sistema.	Algo más de código que MVC (servicios y repositorios separados).	SELECCIONADO, con un conector intercambiable para el núcleo.
Hexagonal	Máxima independencia del núcleo y de la base de datos; muy fácil de probar.	Más clases, interfaces y conceptos (puertos, adaptadores); difícil de sostener sin personal técnico.	Descartado como patrón completo; se toma solo la interfaz del conector.
Microservicios	Escala y despliega cada parte por separado.	Varios servicios que operar y monitorear; exige personal y presupuesto que no existen.	Descartado.

Capas y responsabilidades

Capa	Responsabilidad	Clases principales (paquete)
Seguridad	Rechazar peticiones sin token válido (401) y verificar el rol.	FiltroJwt, ConfiguracionSeguridad (security)
Presentación	Recibir la petición HTTP, validar el formato y devolver el código HTTP correcto. No tiene lógica de negocio.	TransferenciaController, SaldoController, AuditoriaController (controller)
Negocio	Reglas: cuenta autorizada, monto positivo, referencia no repetida, estados de la transferencia y registro en la bitácora.	TransferenciaService, AuditoriaService (service)
Datos	Guardar y leer de PostgreSQL.	TransferenciaRepository, RegistroAuditoriaRepository, … (repository) + entidades (model)
Conector del núcleo	Única parte que habla con el núcleo legado. Tiene dos versiones intercambiables por configuración: la real y el simulador.	NucleoConector (interfaz), NucleoLegadoConector, NucleoSimulador (integration)

Regla de dependencias: cada capa solo llama a la de abajo. El servicio no sabe si habla con el núcleo real o con el simulador; solo conoce la interfaz NucleoConector.

2.2 Pila tecnológica (Tech Stack)

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación (AC / restricción)
Lenguaje backend	Java (OpenJDK)	21 LTS	Node.js 20	Restricción de tiempo: dominio previo del estudiante en Java y Spring Boot. Licencia libre (cero presupuesto). Versión LTS: soporte largo, menos mantenimiento (AC-03).
Framework backend	Spring Boot	3.3.x	Express / Quarkus	AC-03: impone la estructura por capas (@RestController, @Service, @Repository) y permite perfiles de configuración (prod / test) para cambiar el conector por el simulador sin tocar código.
Seguridad	Spring Security + JWT + BCrypt	6.x	Sesiones en servidor	AC-01: el consumidor es otro sistema, sin navegador; el token identifica a quién llama y ese dato queda en la bitácora. Secretos guardados con BCrypt.
Acceso a datos	Spring Data JPA + Hibernate	3.3.x	JDBC / SQL a mano	AC-03: repositorios sin escribir SQL repetitivo; las entidades corresponden 1 a 1 con las tablas. Se acepta perder control fino del SQL.
Migraciones	Flyway	10.x	ddl-auto de Hibernate	AC-01: el esquema queda versionado (V1, V2) y se puede auditar cuándo cambió.
Base de datos	PostgreSQL	16	MongoDB	AC-01: llaves foráneas, CHECK, UNIQUE, transacciones y un trigger que impide modificar la bitácora. Licencia libre (cero presupuesto).
Huella de integridad	SHA-256 (incluido en Java)	JDK 21	Servicio de auditoría en la nube	Cero presupuesto: viene con el lenguaje, sin instalar ni pagar nada. Permite detectar si un registro fue alterado.
Conector del núcleo	Interfaz Java propia + simulador	—	API Gateway comercial	Cero presupuesto (sin licencias) + no hay ambiente de pruebas del núcleo: el simulador permite probar todo sin tocar producción.
Frontend	Ninguno en el corte vertical	—	SPA en React	Alcance: el consumidor es la billetera (sistema a sistema); el auditor usa la API. Postman demuestra el flujo completo.
Pruebas	JUnit 5 + Spring Boot Test + Postman	5.10	Solo pruebas manuales	Verificación de las métricas de AC-01, AC-02 y AC-03.
Despliegue	Docker + Docker Compose	26.x	Nube de pago	Cero presupuesto: corre en un servidor que la cooperativa ya tiene. Reinicio automático de contenedores (cero personal).
Diagramas	PlantUML	1.2024	Herramienta con licencia	Los diagramas son texto: se versionan en Git junto al código.
Control de versiones	Git + GitHub	—	—	Restricción del curso.